Introduction

Every developer who grinds LeetCode long enough eventually hits a "two players take turns" problem, and the first instinct is almost always wrong. Stone Game III is one of the cleanest examples of this trap: the greedy move looks obviously correct, and it quietly loses. That makes it a favorite in interviews — not because the code is long, but because it tests whether you can catch yourself before writing the wrong solution.

This problem also teaches a technique that shows up again and again in interview questions: collapsing two competing scores into a single number. Once you've internalized that trick, an entire category of "two players, one row, take turns" problems stops being scary. That's the real reason to spend time here — not to memorize this one solution, but to recognize the shape of it the next time it shows up wearing a different costume.

Problem Overview

You're given a row of stones, and each stone has a point value — including, sometimes, a negative value, where taking that stone actually hurts your score. Alice and Bob take turns, Alice first. On each turn, the current player must take 1, 2, or 3 stones from the *front* of the row — never from the middle, never from the back. Whatever they take gets added to their running score.

Both players play optimally, meaning neither one ever makes a careless move — every choice is the best available choice given perfect play from both sides going forward. Once every stone is gone, whoever has more points wins. Equal scores mean a tie.

The row can hold up to 50,000 stones, so the solution has to run in roughly linear time. Anything that branches into multiple recursive paths per stone, without reusing prior work, will time out.

Example

Take the row `[1, 2, 3, 7]`.

If Alice greedily grabs the first three piles (1 + 2 + 3 = 6 points), she leaves Bob with just the 7. Bob takes it and finishes with more points than Alice — Bob wins, despite Alice taking three times as many stones.

Now take `[1, 2, 3, 6]`. Play this out by hand: no matter what Alice opens with, the best she can force, against a Bob who also plays perfectly, is a tie.

These two examples establish the whole point of the problem: the number of stones taken is irrelevant. What matters is the total value locked away in the pile you leave behind for your opponent.

Intuition

The natural first instinct is to simulate the game directly: try taking 1 stone and imagine perfect play from there, rewind, try 2, rewind, try 3. This works for small examples, but it tracks two separate running scores — Alice's and Bob's — and that quickly turns into a tangled mess of bookkeeping. Worse, every stone branches into three future paths, so the total number of game states explodes exponentially. Fine for four stones, hopeless for fifty thousand.

The insight that unlocks this problem is realizing you don't need two scores at all. You only need the *difference* between them.

Define `dp[i]` as: starting from stone `i`, how far ahead can whoever's turn it is end up, assuming both players play optimally from here on? Critically, this number never mentions Alice or Bob by name — it's always framed as "the current player" versus "the other player." That means the exact same formula works regardless of whose turn it actually is, because from the perspective of stone `i`, "the current player" is just whoever moves next.

If `dp[i]` is positive, the player moving at position `i` can force themselves into the lead. This is the score-gap trick: replace two scoreboards with one number representing the gap between them.

Brute Force Solution

Idea: Recursively try all three choices (take 1, 2, or 3 stones) at every position, tracking each player's score separately, and pick whichever choice maximizes the current player's final advantage.

Algorithm: 1. At each position, try taking 1, 2, then 3 stones. 2. For each choice, recurse into the remaining row with roles swapped. 3. Compare the two players' final scores at each leaf and propagate the better choice back up.

Advantages: Directly mirrors the rules of the game, so it's easy to reason about correctness.

Disadvantages: No reuse of subproblems, so it recomputes the same "remaining row" scenario over and over. Managing two scores in parallel also makes the code error-prone.

Complexity: Time is exponential, roughly O(3^n), since each position branches three ways. Space is O(n) for the recursion depth. Completely unusable at 50,000 stones.

Optimal Solution

The fix is the score-gap DP described above, computed backward.

Let `dp[i]` represent the best score gap the current player can achieve starting from stone `i`, with `dp[n]` (one slot past the last stone) defined as `0` — an empty row has no gap to fight over.

For each position `i`, the current player has three options: take 1, 2, or 3 stones. For each option: - Add up the values of the stones just taken (their gain this turn). - Subtract `dp` at the position right after the stones taken.

Why subtract? Because after this move, the remaining row becomes the *opponent's* problem, and `dp` at that position is the opponent's best possible lead over whoever plays next — which is now you. Their gain is your loss, so it's subtracted.

Since `dp[i]` depends on `dp[i+1]`, `dp[i+2]`, `dp[i+3]` — positions further to the right — the table has to be filled from the back of the row toward the front.

Position	Meaning	Depends on
`dp[n]`	Empty row	Base case = 0
`dp[i]`	Best gap starting at stone `i`	`dp[i+1]`, `dp[i+2]`, `dp[i+3]`
`dp[0]`	Final answer	Alice's best possible lead over Bob

Python Code

```
from typing import List


def stone_game_iii(stone_value: List[int]) -> str:
    n = len(stone_value)
    dp = [0] * (n + 1)  # dp[n] = empty row, gap is 0

    for i in range(n - 1, -1, -1):
        best = float("-inf")
        take = 0
        for k in range(3):  # take 1, 2, or 3 stones
            if i + k >= n:
                break
            take += stone_value[i + k]
            best = max(best, take - dp[i + k + 1])
```

```
        dp[i] = best

    if dp[0] > 0:
        return "Alice"
    if dp[0] < 0:
        return "Bob"
    return "Tie"
```

Code Walkthrough

- `dp = [0] * (n + 1)` : one slot per stone, plus one extra slot at index `n` for "empty row, nobody left to play." That extra slot is what the very first backward step subtracts against.
- `for i in range(n - 1, -1, -1)` : fills the table from the last stone back to the first, since `dp[i]` needs later slots already computed.
- `best = float("-inf")` : a starting value guaranteed to lose to any real candidate — there's no valid score yet to compare against.
- `for k in range(3)` : tries taking 1, 2, then 3 stones (`k` is 0-indexed, so `k=0` means one stone).
- `if i + k >= n: break` : stops early near the end of the row where fewer than 3 stones remain.
- `take += stone_value[i + k]` : accumulates the value of the stones grabbed this turn.
- `take - dp[i + k + 1]` : this turn's gain minus the opponent's best gap from the resulting position — the core of the score-gap trick.
- `dp[i] = best` : locks in the best of the three choices.
- Final sign check: `dp[0]` is Alice's best possible lead since she moves first. Positive means Alice wins, negative means Bob wins, zero is a tie.

Dry Run

Row: `[1, 2, 3, 7]`

- `dp[4] = 0` (empty row)
- `dp[3]` : only stone `7` is left, so taking it gives `7 - dp[4] = 7` . `dp[3] = 7`
- `dp[2]` : stones `3, 7` remain.
- take 1 (`3`): `3 - dp[3] = 3 - 7 = -4`
- take 2 (`3+7=10`): `10 - dp[4] = 10 - 0 = 10`
- `dp[2] = 10`
- `dp[1]` : stones `2, 3, 7` remain.

- take 1 (`2`): `2 - dp[2] = 2 - 10 = -8`
- take 2 (`2+3=5`): `5 - dp[3] = 5 - 7 = -2`
- take 3 (`2+3+7=12`): `12 - dp[4] = 12`
- `dp[1] = 12`
- `dp[0]` : stones `1, 2, 3, 7` remain.
- take 1 (`1`): `1 - dp[1] = 1 - 12 = -11`
- take 2 (`1+2=3`): `3 - dp[2] = 3 - 10 = -7`
- take 3 (`1+2+3=6`): `6 - dp[3] = 6 - 7 = -1`
- `dp[0] = -1`

`dp[0] = -1` is negative, so **Bob** wins — exactly matching the greedy trap from the introduction, now proven with numbers instead of intuition.

Complexity Analysis

Time: O(n), since each of the `n` positions does a constant amount of work (at most 3 candidate choices).

Space: O(n) for the `dp` array. This can be trimmed to O(1) by keeping only the last three computed values, since `dp[i]` never depends on anything beyond `i+3`, but the array version is clearer to read and still fits comfortably within 50,000 stones.

Alternative Solutions

A top-down (memoized recursion) version expresses the same idea more literally — write a recursive function `dp(i)` that calls itself for `i+1`, `i+2`, `i+3` and caches results with `functools.lru_cache`. It reads closer to the plain-English rules of the game. The downside: Python's recursion has a call-stack limit, and with 50,000 stones, a naive recursive version risks hitting `RecursionError` for no real benefit over the bottom-up loop. The bottom-up version is the one to ship.

Edge Cases

- **Single stone:** Alice takes it; nothing to strategize. `dp[0]` correctly resolves to that stone's value.
- **All negative values:** The formula still works, since it's always comparing "take this vs. take that" — it finds whichever path loses the least.

- **Exact tie:** If both players' best play lands them even, `dp[0]` naturally comes out to `0`, correctly reporting a tie instead of guessing.
- **Maximum size (50,000 stones):** One linear backward pass handles this as safely as a row of four — no recursion, no exponential blowup.

Common Mistakes

1. Tracking Alice's score and Bob's score as two separate values instead of one score-gap number — it works, but turns into a confusing mess to maintain.
2. Forgetting the extra `dp[n]` slot for "empty row" — without it, the very first backward step has nothing valid to subtract.
3. Assuming taking more stones is automatically better — as shown in the opening trap, a large or negative pile left for the opponent (or grabbed by you) changes everything.
4. Filling the DP table left to right instead of right to left — `dp[i]` depends on values that don't exist yet if computed forward.
5. Reading the final sign backward — forgetting that `dp[0]` represents *Alice's* lead since she moves first, not Bob's.

Interview Questions

- Why does this problem require backward iteration instead of forward iteration?
- How would you modify this solution if players could take 1 to `k` stones instead of 1 to 3?
- Can you reduce the space complexity from O(n) to O(1)? Walk through how.
- Why does tracking the score difference work instead of tracking both scores independently?
- What changes if stones can also be taken from the back of the row?

Similar Problems

- **Stone Game (LeetCode 877):** The original version, always solvable with a first-player-wins guarantee for even-length rows — good for seeing the score-gap idea in its simplest form.
- **Stone Game II (LeetCode 1140):** Same score-gap idea, but the take-limit grows as the game progresses, adding a second state dimension.
- **Predict the Winner (LeetCode 486):** Nearly identical structure to Stone Game, taking from either end of the array.
- **Guess Number Higher or Lower II (LeetCode 375):** A different two-player minimax setup that rewards the same "think in terms of guaranteed outcome" mindset.

Key Takeaways

Greedy reasoning fails here because taking more stones now can strand you with a worse position later. The fix isn't a clever formula — it's a reframing: instead of tracking two competing scores, track the single number that represents the gap between them, computed backward so every choice can look ahead at what the opponent will do with what's left. Once that reframing clicks, the DP itself is just three candidate sums and a `max()`.

Watch the Video

For the full hand-traced walkthrough, including the visual buildup of the DP table and the "your turn" example with a negative-value pile, watch the video here: {LINK}

About the Series

This breakdown is part of **Fun with Learning Technology**, a series built around taking one interview-style coding problem at a time and rebuilding it from a naive first instinct into a clean, production-ready solution — with the reasoning shown, not just the answer.

Call To Action

If this made the score-gap trick click, give the video a like on YouTube and subscribe for more of these walkthroughs. For the full written breakdowns delivered straight to your inbox, subscribe to the Substack. And if a problem tripped you up recently, drop it in the comments — it might be the next one covered.

Quick Review

Grabbing the biggest visible pile — why is that greedy move wrong here?

It ignores what the opponent can take next; a smaller pile now can force a worse split for them later.

Stone Game III: what's the naive baseline approach?

Simulate every possible sequence of turns (try all pile-count choices recursively) — brute force before optimizing.

What state does the 'jar of coins' framing hint we need to track?

Whose turn it is and the remaining piles — score difference between the two players, not just total taken.

Why does 'one tiny row' matter for this problem's structure?

The whole game is just one 1D array of piles — no grid/board complexity, only left-to-right choices from the front.

How many pile-taking options does a player have each turn (front three rule)?

Take 1, 2, or 3 piles from the front of the remaining row — never from the middle or end.
